

## **Project 2: A Sparse Matrix Processor**

Discussed: Tuesday, March 24, 2026

Posted: Monday, March 23, 2026

Due date: Friday 11:59 pm April 17/May 1, 2026 (phase 1/phase 2)

### **Project Objectives**

1. Learn separate compilation and linking for large projects.
2. Master the concepts of pointer, pointer array, and dynamic memory allocation.
3. Master basic linked list operations: build, search, insert, delete.
4. Learn the design of complex (more than doubly) linked list for sparse data set.

### **Project Description**

In this project, you will implement a set of operations on sparse matrixes, where majority of the matrix entries are 0's and others are non-zero **integers**. These operations include: addition and multiplication of two matrixes; search, transpose, reverse, and find the maximum entry of a matrix; and find a path between two entries in a matrix.

You will be given a few input files, each representing a sparse matrix; a complete program `p2.c` with the `main()` function; and a header file `sparse.h` with all the function prototypes required in the `main()` function. Your assignment is to implement all the functions defined in `sparse.h` in a file called `sparse.c` such that `p2.c` can compile and run correctly. A sample `sparse.c` with the implementation of a couple of functions will also be provided for you to start with.

The project consists of two phases. Details on the requirements and submission information for each phase are explained in this document.

### **General Requirements**

For this project, your program should run with the following command:

```
a.out matrix1.txt matrix2.txt .. matrixk.txt results.txt
```

where `a.out` is the compiled executable; `matrix1.txt ... matrixk.txt` are the input files, each representing one sparse matrix of unknown size and unknown number of non-zero entries. The number of such input files can go from 1 to any number `k`; `results.txt` is the output file that records the result of each matrix operation in the `main()` function. You can assume that the command line arguments are correct and all the input files are valid without any errors.

### ***Input files:***

Each line in the input file `matrix.txt` represents one non-zero entry in the sparse matrix. For example, if the value at position (12, 305) of the input matrix is 6789, it will be represented as

```
12 305 6789
```

The size of the input files (i.e. the number of non-zero entries in the sparse matrix, the number of rows, and the number of columns) are all unknowns. These entries are also listed in the input file in a random order. In the sample `sparse.c`, a function is implemented to read in the input file and create a singly linked list of these non-zero entries. So you do not need to open these input files, you can just use these linked lists for the rest of the project.

**Output file:**

This file records the matrix operations called in the `main()` function of program `p2.c`. For each operation, you first print the name of the operation and then the result of the operation. Details will be defined below when each matrix operation is defined and specified in `sparse.h`.

**Data structures of the sparse matrix:**

A non-zero valued entry in the sparse matrix is represented by the following structure

```
typedef struct entry
{ int val;                // non-zero integer value of the entry
  int row, col;           // entry's position in the matrix
  struct entry *left, *right; // pointers to the first non-zero entry of the
                              // matrix that is to the left (smaller column
                              // number) or right (larger column number) of
                              // this entry and in the same row of this entry

  struct entry *up, *down; // pointers to the first non-zero entry of the
                              // matrix that is above (smaller row number
                              // or below (larger row number) this entry
                              // and in the same row of this entry

  struct entry *next;      // a pointer to another entry in the matrix
  void * info;             // you can define additional information about
                              // an entry and use this pointer to access
                              // that information
} ENTRY;
```

A ROOT structure is defined below for the list of ENTRY's in the matrix using a singly linked list.

```
typedef struct root
{ ENTRY *head;           // root to a list of all non-zero entries
                          // first entry in a "matrix" or a list of ENTRY's.
                          // You can traverse the list with appropriate
                          // "next" pointer. That is, use "next" to access
                          // the entire matrix.

  long num;              // the number of non-zero entries in the matrix
  void *info;            // a pointer that you can use to define anything
                          // that you find useful
} ROOT;
```

The following two functions are implemented in the sample `sparse.c`

```
void read_input(FILE *F, ROOT *M, FILE *G)
```

This function reads in the non-zero entries from an input file **F** and stores them at ROOT **M**. The following messages will be written to the output file **G**, where the two hexadecimal numbers are the memory locations that pointers **F** and **M** point to, respectively.

```
Reading entry from 0x1afc270 ...
5 entries stored in matrix at 0x1afc610.
```

```
void show_list(ROOT *M, FILE *F)
```

This function prints out the non-zero entries and their position in matrix **M** to output file **F** together with the following message. Each number takes 5 spaces and there are 5 numbers on each line.

```
matrix at 0x10fbcd0 ...
  1 (2,2)    8 (2,3)    7 (2,1)    6 (3,2)    7 (3,1)
  9 (3,3)
```

**Matrix Operations:**

You will implement the following matrix operations in `sparse.c`. Detailed specification of each function together with the output format and requirements will be stated in the function's prototype and can be found in `sparse.h`. All the functions have the output file as a parameter in order to print out the information of the matrix operation.

**1: search**            `void search (ROOT *M, int k, FILE *F)`

This function has two inputs: a matrix and a non-zero integer value  $k$ . It will search in the matrix and print out the position(s) whose entry has value  $k$ . If there are multiple occurrences of  $k$ , print out all those positions.

**2: find\_max**            `void find_max(ROOT *M, FILE *F)`

This function takes a matrix as input and prints out the maximum non-zero value and its position. If there are multiple occurrences of the maximal value, printing out one position is sufficient.

**3: addition**            `ROOT * addition (ROOT *M1, ROOT *M2, FILE *F)`

This function takes two matrices as input and create a new matrix of their sum. For matrices  $A$  and  $B$ , let their entries at position  $(i,j)$ , that is the  $i$ -th row and  $j$ -th column, be  $A_{ij}$  and  $B_{ij}$ , then the value at the same position will be  $A_{ij} + B_{ij}$  for matrix  $A+B$ .

**4: multiplication**    `ROOT * multiplication (ROOT *M1, ROOT *M2, FILE *F)`

This function takes two matrices as input and create a new matrix of their product. For matrices  $A$  and  $B$ , let their entries at position  $(i,j)$  be  $A_{ij}$  and  $B_{ij}$ , then the value at the same position will be  $A_{i1} * B_{1j} + A_{i2} * B_{2j} + \dots + A_{ik} * B_{kj} + \dots$  for  $A * B$ . Note that there is no need to check whether the sizes of two matrices match or not (because we can always add rows or columns with all 0's).

**5: transpose**            `ROOT * transpose (ROOT *M, FILE *F)`

This function takes a matrix  $M$  as input and creates its transpose matrix. Matrix  $A = \{A_{ij}\}$  is the transpose of matrix  $B = \{B_{ij}\}$  if and only if  $A_{ij} = B_{ji}$ .

**6: reverse**            `ROOT * reverse (ROOT *M, FILE *F)`

This function takes a matrix  $M$  as input and creates its reverse matrix. Matrix  $A = \{A_{ij}\}$  is the reverse of matrix  $B = \{B_{ij}\}$  if and only if the non-zero entries of  $A$  and  $B$  are at the same positions, but in the reverse order. That is, if we order the non-zero entries from the first row to the last row and from the first column to the last column in the same row as,  $A = \{a_1, a_2, a_3, \dots, a_{k-2}, a_{k-1}, a_k\}$  and  $B = \{b_1, b_2, b_3, \dots, b_{k-2}, b_{k-1}, b_k\}$ , then  $a_i$  and  $b_i$  are at the same position in the matrix and  $a_1 = b_k, a_2 = b_{k-1}, \dots, a_{k-1} = b_2, a_k = b_1$ .

**7: find\_path**

`int find_path (ROOT *M, int r1, int c1, int r2, int c2, FILE *F)`

This function has three inputs: a matrix, and two entries,  $p$  at  $(r1, c1)$  and  $q$  at  $(r2, c2)$ , in the matrix. A path from  $p$  to  $q$  is defined as a series of entries with non-zero values  $\{r_0, r_1, r_2, \dots, r_k\}$  such that  $r_0 = p, r_k = q$ , and for each pair  $(r_i, r_{i+1})$ ,  $r_i$  and  $r_{i+1}$  are either in the same row or in the same column. Such path may not exist and, when exists, may not be unique. If there are multiple paths from  $p$  to  $q$ , no need to print out all the paths, printing any path will be sufficient.

**Note:** there is a difference between a matrix that does not exist and a matrix that does not have any non-zero entries (or all zeros). All the operations involved former will return nothing or NULL. For the latter, the matrix will be treated as 0 (a matrix with all zeros).

**Submission Requirements:**

1. The project must be implemented using C under the GLUE UNIX system in a file named `sparse.c` which should be submitted the same way you submitted project 1.
2. Your program must be properly documented. In addition to the program header, comments, variable naming, revision history, etc., describe clearly (1) any new data structure you introduced in your implementation of the project, (2) your method to find the reverse of a matrix, (3) your method to find a path between two entries in a sparse matrix, and (4) any new functions that you introduce and implement in `sparse.c`.
3. The goal of the project is to use linked list for sparse matrix operations, you CANNOT use data structures such as 2-dimensional array to represent the matrices and perform the matrix operations.
4. A `p2.c` file, a `sparse.c` file and the `sparse.h` file will be posted in the GLUE public directory. You must implement all the functions declared in `sparse.h` in your `sparse.c` such that `p2.c` can run correctly. You CANNOT make any changes to `p2.c` or `sparse.h`.
5. The project is designed in two phases. In phase 1, your main job is to design data structures (if needed) for the matrix operations and implement functions search, find\_max, and addition; in phase 2 you will focus on implementing multiplication, transpose, reverse, and find\_path. For the two different phases, files `p2_1.c` and `p2_2.c` will be used as `p2.c`. The two submission deadlines are
  - a. April 17: phase 1, file to submit: `sparse_1.c` (50 points)
  - b. May 1: phase 2, file to submit: `sparse_2.c` (50 points)

**Grading Criteria:**

- 10% Correct compilation
- 80% Correct execution
- 10% Coding style and documentation

Files that cannot be compiled will be given a 0.

Late penalty is 20% per day. Submissions more than two days late will not be accepted.